

Dynesty.jl 综合测试与基准报告

Dynesty.jl migration test report

2026-06-11

摘要

本报告汇总了当前 Dynesty.jl 工作树在 Julia 默认测试套件、示例脚本、parallel map backend、Python fixture 兼容性、benchmark runner、Python 脚本语法检查、formal parallel cost benchmark 审计、真实空气质量 PE documented scaled formal benchmark, 以及 LaTeX 编译链路上的验证结果。常规 Julia 测试通过 `julia --project=. -e 'using Pkg; Pkg.test()'` 全量执行并通过; formal parallel cost benchmark 复用已经完成的 `examples/output/parallel_cost_compare` 输出, 审计确认其为 `mode == "formal"`、18 个 run 全部 ok、9 个 `corner_overlay` posterior overlay plot 全部 ok, 且所有 run 均使用 `/usr/bin/time -v` 并记录 process-tree RSS/PSS 内存字段。新增空气质量 benchmark 使用 synthetic PM2.5 sensor calibration/source-strength inverse problem, likelihood 仅在 Julia 中实现, Julia runner 直接调用该 likelihood, Python dynesty 通过 PyJulia bridge 调用同一个 Julia likelihood; 当前环境通过临时 mount namespace 暴露 GNU `/usr/bin/time -v`, 完成 `nlive=500`、`dlogz=0.08`、2 repeats、31 workers 的 documented scaled formal run, 并生成 posterior overlay 图。

目录

1	测试环境	3
2	验证命令矩阵	4
3	迁移与兼容性测试概览	6
4	Formal Benchmark 审计	6
5	Parallel Cost Benchmark 结果	7

目录	2
6 Posterior Overlay Plots	9
7 真实空气质量 PE Benchmark	12
8 结论与剩余风险	17
A 全部 Benchmark Run	17
B 全部 Posterior Overlay Plot	19

1 测试环境

本报告基于当前工作树生成，未修改只读参考仓库 `../dynesty`，也未重跑 formal benchmark。formal benchmark 输出创建时间为 2026-06-11T11:14:00.588，其配置记录在 `examples/output/parallel_cost_compare/summary.json`。

项目	值与说明
Julia	1.12.6; <code>Threads.nthreads() == 32</code> ; <code>BLAS.get_num_threads() == 1</code> 。
Python 平台	3.9.18; executable: <code>/home/czc/opt/miniconda3/bin/python3</code> 。 Linux x86_64; Linux 6.1.174-1-MANJARO; AMD Ryzen 9 5950X 16-Core Processor; 32 logical threads。
当前 JULIA_NUM_THREADS	当前 shell 为 32; formal benchmark 的 Julia run 使用 <code>--threads=31</code> 。
OPENBLAS_NUM_THREADS	formal benchmark summary 记录为 1; 当前 shell 探针为 1。
OMP_NUM_THREADS	formal benchmark summary 记录为 1; 当前 shell 探针为 unset。
MKL_NUM_THREADS	formal benchmark summary 记录为 1; 当前 shell 探针为 1。
Python dynesty source path	<code>../dynesty/py</code> ; 解析到 <code>/home/czc/projects/working/dynesty/py</code> ; 只读引用。
正式 parallel benchmark	Julia 31 threads vs Python 31 multiprocessing worker processes; <code>queue_size=31</code> , <code>nlive=1500</code> , <code>dlogz=0.02</code> 。
空气质量 PE scaled formal	Julia 31 threads direct Julia likelihood vs Python dynesty 31 worker processes through PyJulia bridge; <code>queue_size=31</code> , <code>nlive=500</code> , <code>dlogz=0.08</code> , 2 repeats; direct Julia likelihood median 约 0.0091s。
当前 <code>/usr/bin/time</code>	系统全局 <code>/usr/bin/time</code> 与 <code>/bin/time</code> 不存在; 本次空气质量 formal run 在临时 mount namespace 中把编译的 GNU time 暴露为 <code>/usr/bin/time</code> ，未修改系统 <code>/usr/bin</code> 。

表 1: 测试与 benchmark 环境摘要。

Python 参考快照来自 `docs/source_snapshot.md`: `../dynesty commit` 前缀为 `3ec158de0d2b`，完整 commit hash 记录在 source snapshot 中; branch 为 `master`，快照时 `git status --short` 为 `clean`，Python dynesty 版本为 2.1.2。本报告没有对 `../dynesty` 执行修改、pull 或 branch switch。

2 验证命令矩阵

表 2 汇总本次报告生成前后执行的关键检查。耗时仅记录从测试输出或命令上下文可直接取得的信息；没有可靠墙钟秒数的短命令用“未单独计时”标注，避免伪造精确耗时。

test name	command	status	duration	notes
Julia package/test suite	<code>julia --project=. -e 'using Pkg; Pkg.test()'</code>	ok	未单独计时	默认 <code>test/runtests.jl</code> 全量通过, 包含 <code>parallel</code> 、 <code>static/dynamic sampler</code> 、 <code>examples</code> 、 <code>Python fixtures</code> 、 <code>plotting helpers</code> 、 <code>Aqua</code> 。
Examples tests	同上	ok	17.0s	<code>Example scripts</code> : 35 pass。
Parallel tests	同上	ok	多个 testset	<code>Map backends</code> 、 <code>PoolUsage</code> 、 <code>proposal/evolve queue</code> 、 <code>static/dynamic parallel backend tests</code> 均通过。
Benchmark runner include	<code>julia --project=. -e 'include("benchmark/parallel_cost_compare.jl"); println("runner ok")'</code>	ok	未单独计时	输出 <code>runner ok</code> 。
Air-quality likelihood include	<code>julia --project=. -e 'include("examples/air_quality_likelihood.jl"); println("air quality likelihood ok")'</code>	ok	未单独计时	输出 <code>air quality likelihood ok</code> ; <code>calibration helper</code> 可运行。
Air-quality runner include	<code>julia --project=. -e 'include("benchmark/air_quality_pe_compare.jl"); println("air quality runner ok")'</code>	ok	未单独计时	输出 <code>air quality runner ok</code> 。
Python py_compile	<code>python3 -m py_compile benchmark/parallel_cost_corner.py examples/pe_parallel_python.py examples/air_quality_pe_python.py benchmark/air_quality_corner_overlay.py test/reference/python/generate_reference.py</code>	ok	未单独计时	无输出, exit code 0; 未保留 <code>__pycache__</code> 。
Formal benchmark audit	读取并审计 <code>examples/output/parallel_cost_compare/summary.json</code>	ok	未单独计时	<code>mode=formal</code> ; 18/18 run ok; 9/9 plot ok; <code>corner_overlay</code> ; 全部使用 <code>/usr/bin/time -v</code> ; <code>RSS/PSS</code> 字段存在。
Air-quality PE scaled formal	<code>/tmp/dynesty-air-quality-time-src/with-gnu-time.sh julia --project=. benchmark/air_quality_pe_compare.jl --mode formal --repeats 2 --nlive 500 --dlogz 0.08 --queue-size 31 --nproc 31 --work-repeats 64 --calibration-trials 5</code>		4 runs	Python <code>dynesty</code> 通过 <code>PyJulia bridge</code> 调用同一个 Julia likelihood; 2 张 <code>posterior overlay</code> 生成成功; 所有 run 均使用 <code>/usr/bin/time -v</code> 并记录 <code>RSS/PSS</code> 。
LaTeX compile check	<code>latexmk -xelatex -interaction=nonstopmode -halt-on-error main.tex</code>	ok	见编译日志	本机 <code>xelatex/latexmk/ctexart</code> 可用, 生成 <code>docs/test_report/main.pdf</code> 。
Git whitespace check	<code>git diff --check</code>	ok	未单独计时	提交前运行, 确认无 <code>whitespace error</code> 。

表 2: 测试与质量检查矩阵。

3 迁移与兼容性测试概览

当前迁移测试策略由 `docs/testing.md`、`docs/migration_matrix.md`、`docs/compatibility.md` 与 `test/reference/python/README.md` 共同描述。默认 CI 和本次 `Pkg.test()` 不依赖 live Python 或相邻 `../dynesty`，而是读取已经提交的 JSON/NPZ fixture。fixture 记录源 commit、Python/NumPy/SciPy 版本、容差和索引约定；带索引的 fixture 显式记录 Python 0-based 与 Julia 1-based 值。

- Grade A 覆盖 public API 和核心数值函数，要求直接 Julia 测试，并在有意义时使用 Python fixture cross-check。
- Grade B 覆盖影响算法行为的 internal helper，要求直接或间接 Julia 测试；稳定输入输出场景使用 fixture。
- Grade C 覆盖 thin wrappers、display、plotting helpers、compatibility aliases 和 demo/doc helper，通过 caller tests、smoke tests 或迁移矩阵说明覆盖。

本次报告没有实现或替换新的 Python symbol，因此没有为了报告而修改 `docs/migration_matrix.md`。兼容性差异也未发现新的未记录公共行为差异；既有差异包括 bang-form mutating API、1-based dimension indices、`copy_inputs=false` 默认行为、`res.blobs` 字段、Julia-native map backends 与 `PoolUsage`、Julia Serialization/JLD2/HDF5 persistence，以及 `RecipesBase` plotting 等，均已记录在 `docs/compatibility.md`。

未运行的扩展项主要是需要显式环境变量的测试：

- `DYNESTY_RUN_SLOW_TESTS=true`
- `DYNESTY_RUN_PLOT_TESTS=true`
- `DYNESTY_RUN_EXTENDED_TESTS=true`
- `DYNESTY_RUN_DISTRIBUTED_TESTS=true`
- `DYNESTY_RUN_JET_TESTS=true`

这些测试不属于默认 `Pkg.test()` 的 blocking 范围，本次报告将其标记为 not run，而不是把默认测试结果外推为扩展测试结果。

4 Formal Benchmark 审计

按照报告目标，formal benchmark 优先复用已有输出。本次没有重新运行 formal benchmark，因为 `summary.json`、`summary.csv`、`plot_index.csv` 和 9 张 `*julia_vs_python.png` 都存在，且审计结果足以支撑报告结论。审计结果见表 3。

check	expected	observed	status
mode	formal	formal	ok
runs	18 ok	18/18 ok	ok
plots	9 ok	9/9 ok	ok
plot method	corner_overlay	corner_overlay	ok
/usr/bin/time -v	true	true	ok
RSS/PSS fields	present	true	ok
overlay PNG	9 files	9 files	ok

表 3: formal parallel cost benchmark 输出审计。

benchmark 配置为 `nlive=1500`、`dlogz=0.02`、`queue_size=31`、`sleep_ms=0`。Julia run 使用 31 threads 与 `proposal_scheduler=auto`；Python run 使用 `multiprocessing.Pool` 与 31 worker processes。三种 likelihood cost 的 work size 分别为 `cheap=0`、`medium=2000`、`heavy=10000`。所有 run 的 `used_usr_bin_time` 均为 true，同时保留 `time_max_rss_kb`、`process_tree_peak_rss_kb` 和 `process_tree_peak_pss_kb` 等字段。

5 Parallel Cost Benchmark 结果

表 4 从 `summary.json/summary.csv` 提取 cheap、medium、heavy 三种 likelihood cost 下 Julia/Python 的 wall time、CPU utilization、RSS、PSS、nsamples、logz/logzerr。每个 cost 与 implementation 有 3 次 repeat；表中 wall time 同时给出 median 与 min-max range。

cost	impl	runs	median wall s	wall range s	median CPU util.	median RSS KB	median PSS KB	median nsamples	median logz	median logzerr
cheap	julia	3	14.38	14.11–14.47	3.04	1,027,952	942,168	17,392	-6.6755	0.0603
cheap	python	3	3.82	3.60–4.53	2.16	1,602,416	213,612	17,467	-6.7223	0.0609
medium	julia	3	14.39	14.36–14.43	2.97	986,556	901,831	17,471	-6.7300	0.0608
medium	python	3	37.34	37.26–38.08	9.63	1,608,788	218,819	17,527	-6.7628	0.0608
heavy	julia	3	14.90	14.86–14.93	3.54	1,007,436	922,647	17,444	-6.7122	0.0607
heavy	python	3	43.50	41.44–49.27	39.93	1,606,440	218,415	17,391	-6.6732	0.0603

表 4: Julia/Python formal parallel cost benchmark 汇总。CPU utilization 为 total CPU seconds / wall time; RSS/PSS 均为 process tree peak KB。

RSS 与 PSS 的口径不同。RSS (Resident Set Size) 按进程统计驻留物理页, 并在多进程场景中容易把共享页在多个进程上重复计数; 因此 Python multiprocessing 的 process-tree RSS 往往显著偏高。PSS (Proportional Set Size) 会按共享页的进程数分摊内存, 更适合跨单进程多线程 Julia 与多进程 Python 的 process tree 比较。本报告在讨论内存时优先查看 PSS, 同时保留 RSS 以便与常见系统工具口径对应。

从中位数看, cheap cost 下 Python wall time 为 3.82s, Julia 为 14.38s; medium cost 下 Julia 为 14.39s, Python 为 37.34s; heavy cost 下 Julia 为 14.90s, Python 为 43.50s。PSS 中位数方面, Julia 三组约为 942,168 KB、901,831 KB、922,647 KB; Python 三组约为 213,612 KB、218,819 KB、218,415 KB。RSS 中位数方面, Python 三组约为 1,602,416 KB、1,608,788 KB、1,606,440 KB, 受 multiprocessing 共享页重复计数影响, 不能单独作为跨执行模型内存优劣结论。

附录表 7 列出全部 18 个 run 的逐次结果, 以便追踪 repeat-level 波动和 evidence 数值。

6 Posterior Overlay Plots

本报告从 benchmark plot 输出目录复制 9 张 *julia_vs_python.png 到 docs/test_report/figures/, 避免提交 examples/output/。每张图比较同一 cost 与 repeat 下 Julia threaded 与 Python dynesty pool 的 posterior samples 叠加; 蓝色为 Dynesty.jl threaded, 橙色为 Python dynesty pool, 黑色 marker 为 true value。

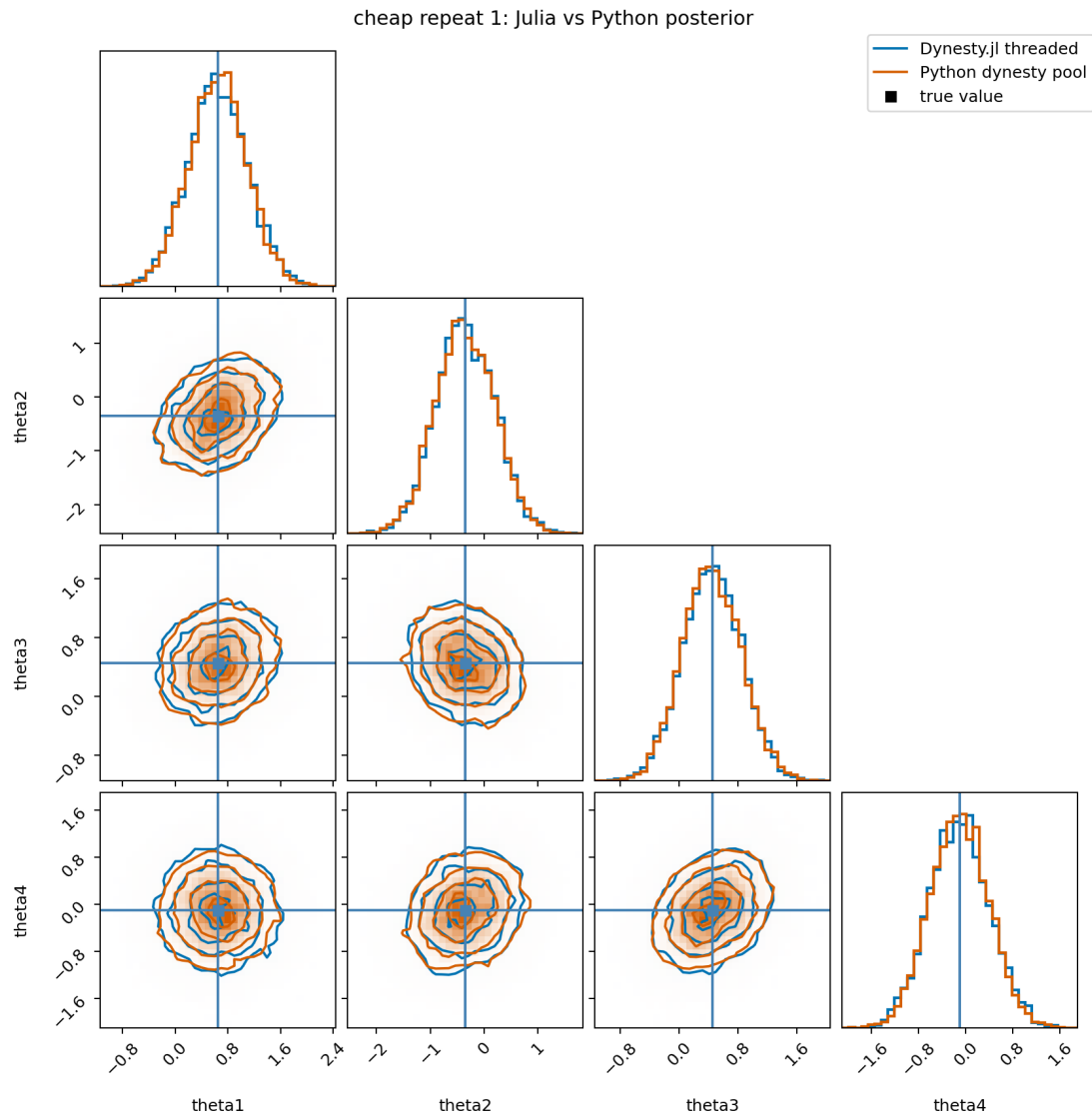


图 1: cheap repeat 1 posterior overlay。蓝色为 Dynesty.jl threaded, 橙色为 Python dynesty pool, 黑色 marker 为 true value。

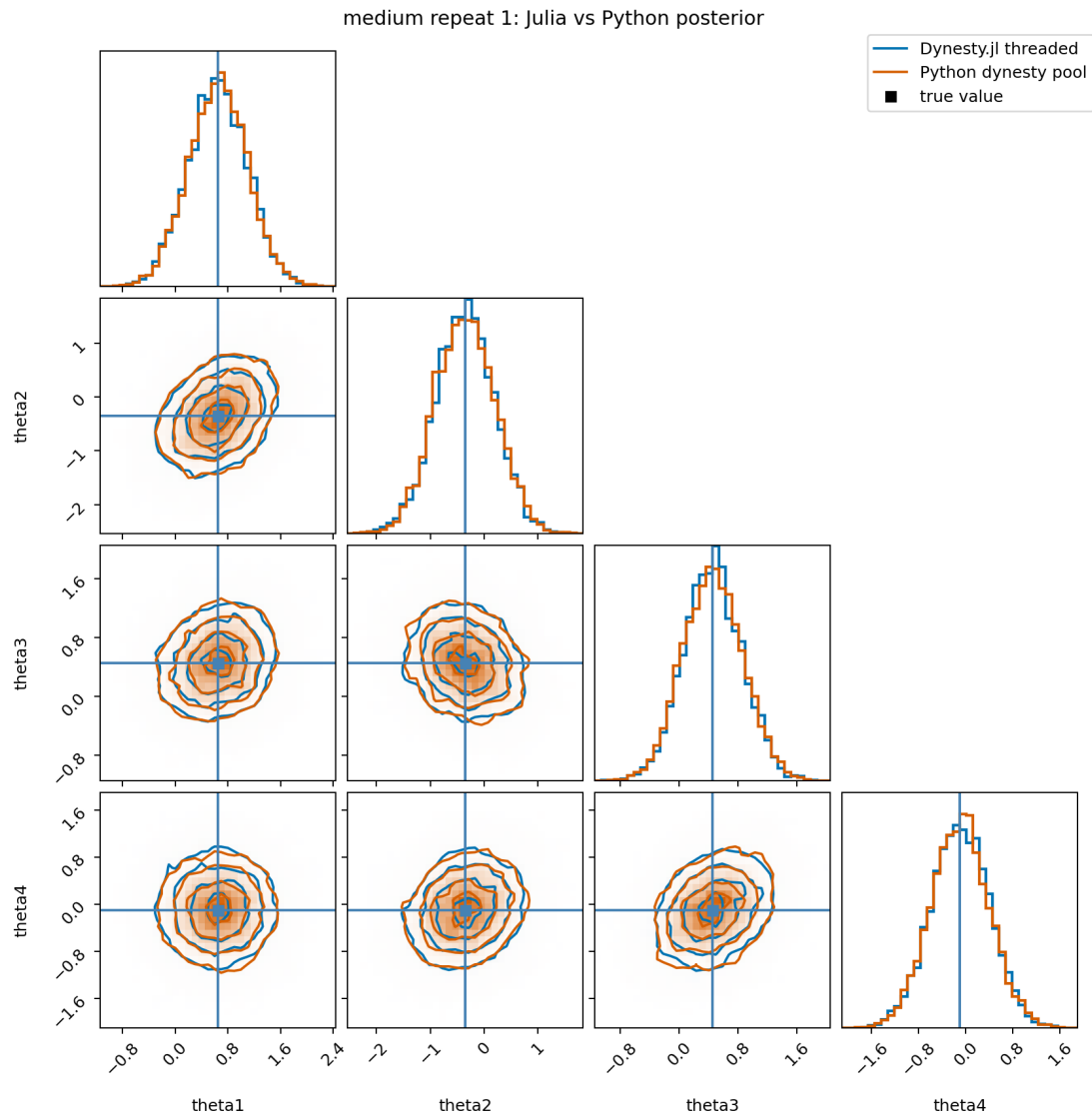


图 2: medium repeat 1 posterior overlay。蓝色为 Dynesty.jl threaded, 橙色为 Python dynesty pool, 黑色 marker 为 true value。

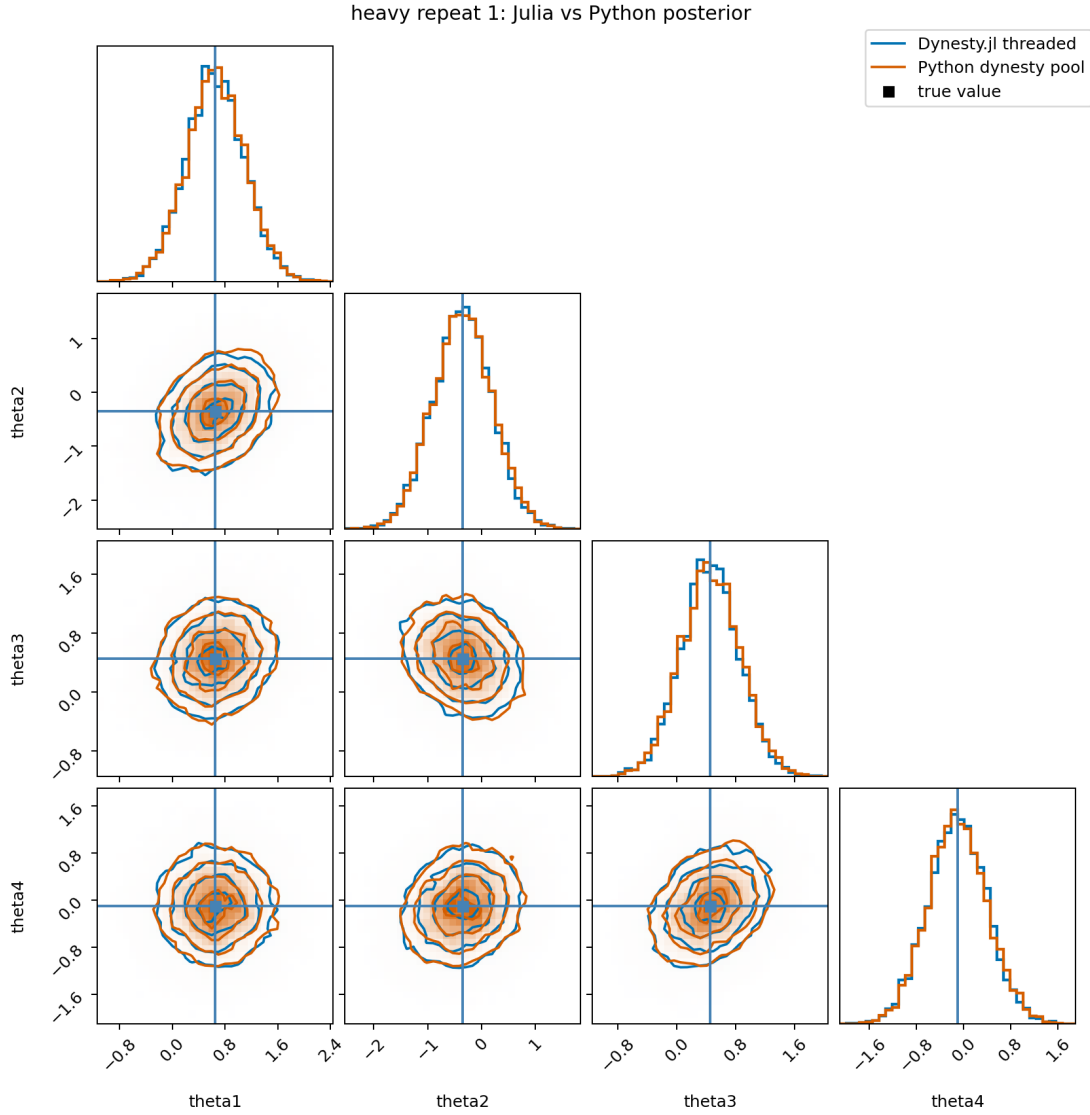


图 3: heavy repeat 1 posterior overlay。蓝色为 Dynesty.jl threaded, 橙色为 Python dynesty pool, 黑色 marker 为 true value。

全部 9 张 overlay 图见附录表 8; 其文件均已复制到报告目录并纳入提交范围。

7 真实空气质量 PE Benchmark

新增 benchmark 位于 `examples/air_quality_likelihood.jl`、`examples/air_quality_pe_julia.jl`、`examples/air_quality_pe_python.py`、`benchmark/air_quality_pe_compare.jl` 和 `benchmark/air_quality_corner_overlay.py`。模型是 deterministic synthetic air-quality inverse problem: 56 个 PM2.5 sensors、48 个 hourly time slices、固定 synthetic source location, 反演 `source_strength`、`background_pm25`、`wind_decay`、`diffusion_length`、`global_bias` 和 `log_noise_scale` 共 6 个参数。观测由固定 seed 生成, 不包含真实个人或敏感数据; 代码和 metadata 明确说明该 synthetic model 不用

于实际环境监管决策。

likelihood 是唯一 canonical implementation: `examples/air_quality_likelihood.jl` 中的 Julia `AirQualityPE.air_quality_loglikelihood`。Julia `Dynesty.jl` runner 直接调用该 Julia likelihood; Python `dynesty` runner 从只读 `../dynesty/py` 导入 Python `dynesty`, 并通过 Python-Julia bridge 调用同一个 Julia likelihood, 没有实现 Python duplicate likelihood。当前环境没有 `juliacall`, 可用 fallback 为 `PyJulia`; 由于本机 Python/PyCall 组合需要 `compiled_modules=false`, runner 在 metadata 中记录 `bridge_kind=pyjulia`, `bridge_compiled_modules=false`, 并且必须先初始化 Julia bridge 再导入 `numpy/dynesty`, 否则 `PyJulia` 会在加载 `PyCall` 时失败。

本节结果来自 documented scaled formal run。配置为 `nlive=500`, `dlogz=0.08`, `repeats=2`, `queue_size=31`, Julia `--threads=31`, Python `nproc=31`, 并设置 `OPENBLAS_NUM_THREADS=1`, `OMP_NUM_THREADS=1`, `MKL_NUM_THREADS=1`。likelihood calibration 选择 `work_repeats=64`, Julia direct median 为 0.00906s, 满足 0.005–0.02 秒目标区间; Python bridge median 中位数为 0.00897s。全局系统缺失 `/usr/bin/time`, 因此本次 run 在临时 mount namespace 中把编译的 GNU time 暴露为 `/usr/bin/time`; benchmark metadata 记录 `time_command_available=true`, `used_usr_bin_time=true`。Linux `/proc` process-tree monitor 同时记录了 RSS/PSS peak, 采样间隔为 0.2s。

check	expected	observed	status
mode	formal for final gate	formal	ok
runs	Julia/Python ok	4/4 ok	ok
plots	overlay ok	2/2 ok	ok
overlay PNG	present	2 files	ok
<code>/usr/bin/time -v</code>	available and used	available=True, used=True	ok
direct likelihood median	0.005–0.02 s	0.00906	ok
work repeats	recorded	64	ok

表 5: 真实空气质量 PE documented scaled formal benchmark 审计。

impl	status	wall s	RSS KB	PSS KB	workers	bridge	Julia like s	bridge like s	nsamples	logz	logzerr
julia	ok	24.33	856,988	744,111	31	–	0.00894	–	11,488	-6950.7386	0.2184
julia	ok	25.88	868,236	755,969	31	–	0.00918	–	11,447	-6950.5997	0.2176
python	ok	23.68	22,909,368	16,206,833	31	pyjulia	–	0.00885	11,312	-6950.3518	0.2169
python	ok	39.94	23,402,288	16,694,658	31	pyjulia	–	0.00909	11,252	-6950.2700	0.2162

表 6: 真实空气质量 PE documented scaled formal run 结果。wall time 和 CPU utilization 来自 `/usr/bin/time -v`。

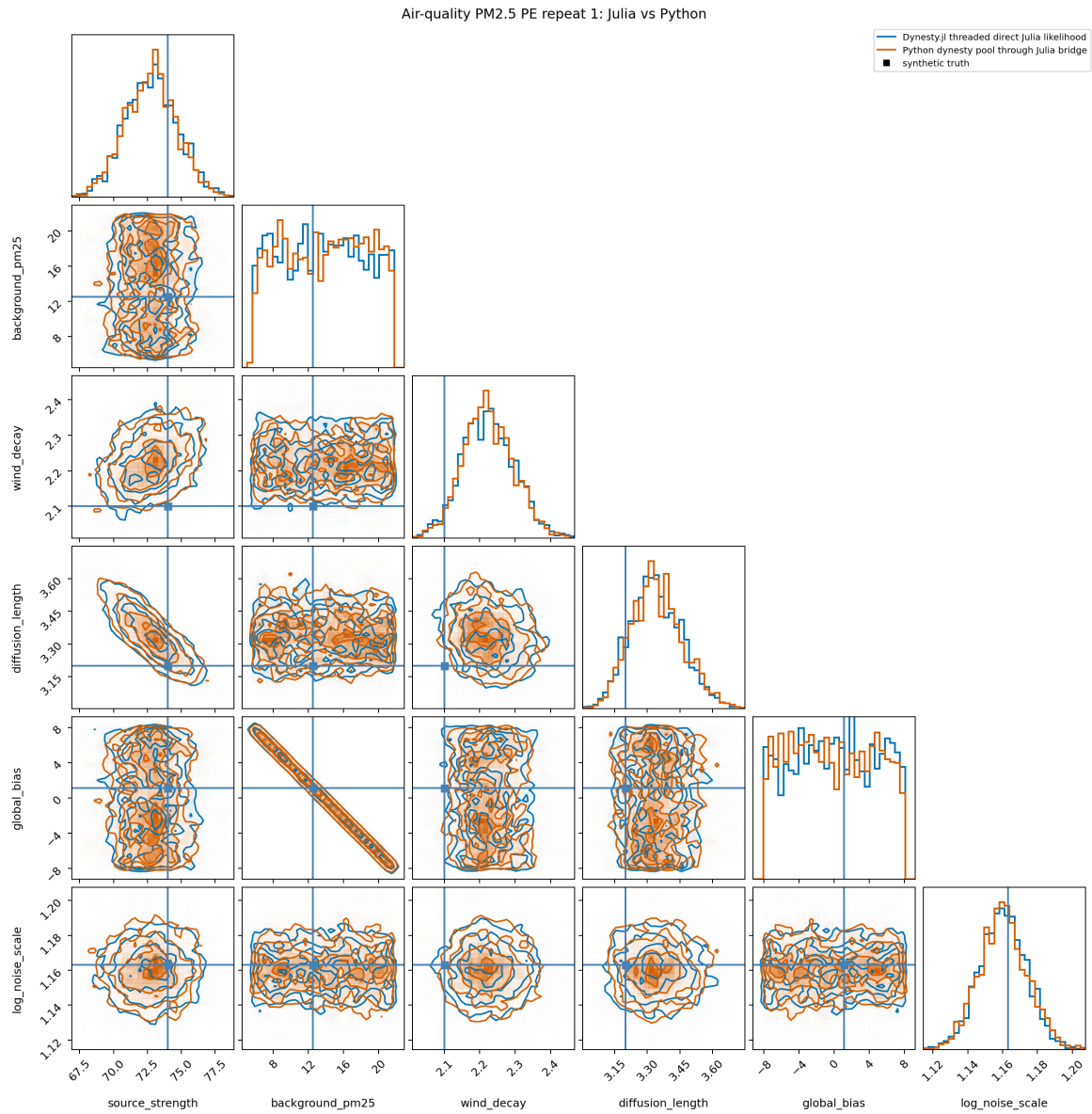


图 4: 真实空气质量 PM2.5 PE repeat 1 posterior overlay。蓝色为 Dynesty.jl threaded direct Julia likelihood; 橙色为 Python dynesty pool through Julia bridge; 黑色 marker 为 synthetic truth。

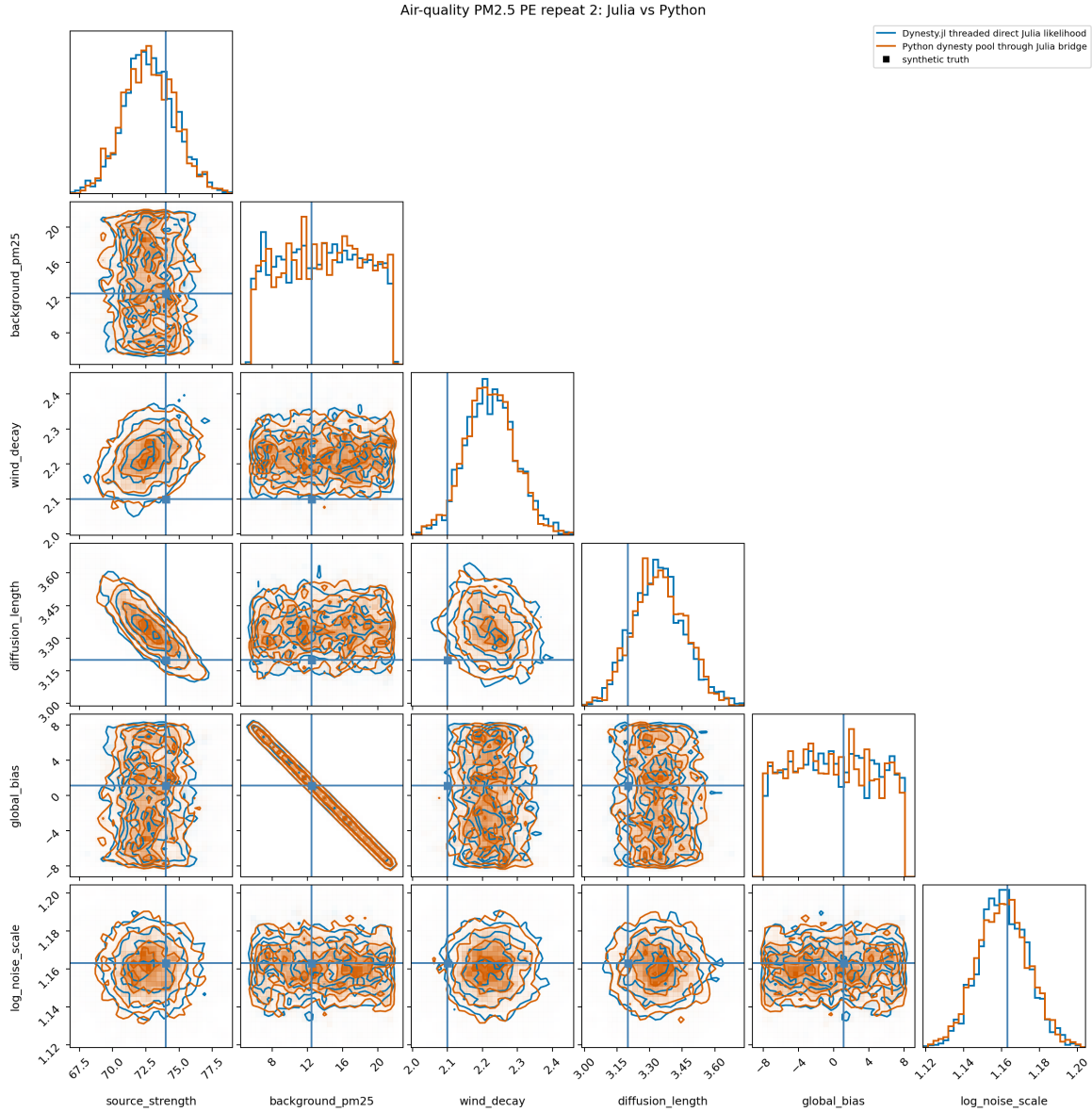


图 5: 真实空气质量 PM2.5 PE repeat 2 posterior overlay。蓝色为 Dynesty.jl threaded direct Julia likelihood; 橙色为 Python dynesty pool through Julia bridge; 黑色 marker 为 synthetic truth。

Julia 两次 run 均为 `status=ok`, wall time 为 24.33s 和 25.88s, process-tree peak PSS 为 744,111 KB 和 755,969 KB, nsamples 分别为 11488 和 11447。Python 两次 run 均为 `status=ok`, wall time 为 23.68s 和 39.94s, process-tree peak PSS 为 16,206,833 KB 和 16,694,658 KB, nsamples 分别为 11312 和 11252。Python wall time 包含 PyJulia main process 和 worker process initialization; 主进程 bridge initialization metadata 记录为 30.59s 和 42.96s。RSS/PSS 差异受 Python multiprocessing、每个 worker 独立 Julia runtime、共享页计数和当前 PyJulia fallback 配置影响。

8 结论与剩余风险

本次常规 Julia package/test suite 全量通过, 且 `test/runtests.jl` 覆盖了 `parallel`、`examples`、`Python fixtures`、`plotting helpers` 和 `Aqua quality checks`。benchmark runner include check 与 `Python py_compile` 也通过。formal benchmark 没有重跑, 而是复用已有 `examples/output/parallel_cost_compare` 输出; 审计确认该输出完整、一致, 并且记录了 `/usr/bin/time -v` 与 `process-tree RSS/PSS` 指标。

posterior overlay 图在 cheap、medium、heavy 三种 cost 的 3 次 repeat 上全部生成成功; 代表图和附录列表显示 Julia/Python 后验叠加比较已经可用于可视化层面的一致性检查。新增空气质量 PE documented scaled formal 也已生成 2 张 Julia/Python posterior overlay, 且 Python dynesty 确认通过 PyJulia bridge 调用同一个 Julia likelihood。性能方面, 当前 parallel cost benchmark 下 Python multiprocessing 在 cheap likelihood 上 wall time 更低; 当 likelihood cost 提升到 medium/heavy 后, Julia threaded 的 wall time 中位数更低。空气质量 benchmark 中 Julia median wall time 为 25.10s, Python PyJulia path median wall time 为 31.81s, 但该结论包含 PyJulia initialization、31 个 worker runtime 和当前 synthetic model 配置。

剩余风险包括: nested sampling 与 posterior visualization 受随机性和有限 repeat 数影响; 性能结论依赖 AMD Ryzen 9 5950X、当前 Julia/Python/BLAS/线程设置和操作系统调度; Python multiprocessing memory accounting 的 RSS/PSS 解释依赖 Linux `/proc` 口径; 空气质量 benchmark 是 prompt 允许的 documented scaled formal, 而不是 800–1500 nlive、3 repeats 的完整目标规模; PyJulia fallback 使用 `compiled_modules=false`, 初始化开销和 worker memory 与 `juliacall` 或 `python-jl` 路径可能不同; 扩展测试 flags 未在本次默认报告中打开; future benchmark 输出若更新, 应重新运行 `docs/test_report/generate_assets.jl` 并重新编译 PDF。

A 全部 Benchmark Run

cost	impl	rep	wall s	CPU util.	RSS KB	PSS KB	nsamples	logz	logzerr	status
cheap	julia	1	14.38	3.04	1,027,952	942,168	17,412	-6.68684	0.06032	ok
cheap	julia	2	14.11	3.00	1,037,576	951,968	17,365	-6.66373	0.06024	ok
cheap	julia	3	14.47	3.06	1,022,420	937,667	17,392	-6.67551	0.06041	ok
cheap	python	1	4.53	1.86	1,601,440	213,612	17,404	-6.68693	0.06067	ok
cheap	python	2	3.60	2.32	1,602,416	215,042	17,566	-6.79324	0.06101	ok
cheap	python	3	3.82	2.16	1,602,688	212,428	17,467	-6.72226	0.06086	ok
medium	julia	1	14.39	3.12	967,948	883,097	17,389	-6.67271	0.06048	ok
medium	julia	2	14.43	2.97	999,464	913,747	17,526	-6.76417	0.06087	ok
medium	julia	3	14.36	2.90	986,556	901,831	17,471	-6.73004	0.06076	ok
medium	python	1	37.26	9.72	1,608,788	217,776	17,544	-6.77774	0.06097	ok
medium	python	2	37.34	9.56	1,611,076	219,379	17,404	-6.68747	0.06063	ok
medium	python	3	38.08	9.63	1,601,988	218,819	17,527	-6.76280	0.06078	ok
heavy	julia	1	14.90	3.42	1,033,952	948,655	17,502	-6.74917	0.06088	ok
heavy	julia	2	14.86	3.54	992,624	906,797	17,444	-6.71222	0.06067	ok
heavy	julia	3	14.93	3.54	1,007,436	922,647	17,416	-6.69062	0.06064	ok
heavy	python	1	49.27	35.78	1,606,440	218,759	17,425	-6.69573	0.06045	ok
heavy	python	2	43.50	39.93	1,610,588	216,908	17,391	-6.67325	0.06032	ok
heavy	python	3	41.44	42.17	1,605,772	218,415	17,291	-6.60632	0.06007	ok

表 7: 18 个 formal benchmark run 的逐次结果。

B 全部 Posterior Overlay Plot

cost	rep	file	method	status	Julia n	Python n	Julia logz	Python logz
cheap	1	corner_cheap_repeat1_julia_vs_python.png	corner_overlay	ok	17,412	17,404	-6.68684	-6.68693
cheap	2	corner_cheap_repeat2_julia_vs_python.png	corner_overlay	ok	17,365	17,566	-6.66373	-6.79324
cheap	3	corner_cheap_repeat3_julia_vs_python.png	corner_overlay	ok	17,392	17,467	-6.67551	-6.72226
medium	1	corner_medium_repeat1_julia_vs_python.png	corner_overlay	ok	17,389	17,544	-6.67271	-6.77774
medium	2	corner_medium_repeat2_julia_vs_python.png	corner_overlay	ok	17,526	17,404	-6.76417	-6.68747
medium	3	corner_medium_repeat3_julia_vs_python.png	corner_overlay	ok	17,471	17,527	-6.73004	-6.76280
heavy	1	corner_heavy_repeat1_julia_vs_python.png	corner_overlay	ok	17,502	17,425	-6.74917	-6.69573
heavy	2	corner_heavy_repeat2_julia_vs_python.png	corner_overlay	ok	17,444	17,391	-6.71222	-6.67325
heavy	3	corner_heavy_repeat3_julia_vs_python.png	corner_overlay	ok	17,416	17,291	-6.69062	-6.60632

表 8: 全部 9 张 Julia-vs-Python posterior overlay plot。